Sure. I'll **simulate one full run** of your *sanity search* script with a **tiny fake dataset** (3 items) and show **what each variable holds** as we go **function-by-function**.

I'll assume:

- `INDEX_VERSION = "20260204_0625"`
- `TOP_K = 2`
- Query = `"veg burger under 200"`

✅ Function 1: `load_artifacts(index_version)`

Inputs

```
index_version = "20260204_0625" INDEX_DIR = "./indexes"
```

Paths created

```
index_path = "./indexes/faiss_20260204_0625.index" mapping_path =
"./indexes/mapping_20260204_0625.json" metadata_path =
"./indexes/metadata_20260204_0625.json"
```

What these files *contain* (simulated)

`vector_map` **(loaded from mapping JSON)**

```
vector_map = { "0": "id_aaa", "1": "id_bbb", "2": "id_ccc" }
```

Meaning:

- FAISS vector 0 corresponds to DB `item_id = id_aaa`
- FAISS vector 1 → `id_bbb`
- FAISS vector 2 → `id_ccc`

`metadata` **(loaded from metadata JSON)**

```
metadata = { "index_version": "20260204_0625", "embedding_model": "all-MiniLM-L6-v2",
"item_count": 3, "dimension": 384, "phase": "phase1", "similarity": "cosine" }
```

`index` **(FAISS index object)**

```
index = <faiss.IndexFlatIP object> # (internally holds 3 vectors)
```

Return values

```
return index, vector_map, metadata
```

So now in `main()` you have:

```
index = <faiss IndexFlatIP> vector_map = {"0":"id_aaa","1":"id_bbb","2":"id_ccc"}
metadata = {...}
```

✅ Function 2: `main()` (top part)

Variable states

```
INDEX_VERSION = "20260204_0625"
```

Console output (simulated)

```
🔹 Loading artifacts for version: 20260204_0625
✅ Metadata: { ...metadata printed... }
```

Then:

```
model = SentenceTransformer(EMBEDDING_MODEL_NAME)
```

Now:

```
model = <SentenceTransformer all-MiniLM-L6-v2>
```

✅ Step: Query embedding

```
query = "veg burger under 200"
```

Console:

```
🔎 Query: veg burger under 200
```

Encode:

```
q_vec = model.encode([query], normalize_embeddings=True)
```

Simulated `q_vec` (shape is `(1, 384)` but we'll show small):

```
q_vec = [ [0.12, -0.03, 0.08, ... 384 dims ...] ]
```

Then:

```
q_vec = np.asarray(q_vec, dtype="float32")
```

Now:

```
q_vec = array([[0.12, -0.03, 0.08, ...]], dtype=float32) q_vec.shape = (1, 384)
```

✅ Step: FAISS search

```
scores, vector_ids = index.search(q_vec, TOP_K)
```

Simulated FAISS result:

```
vector_ids = [[1, 0]] scores = [[0.88, 0.82]]
```

Meaning:

- Best match is vector id **1** (score 0.88)
- Second match is vector id **0** (score 0.82)

Then your code does:

```
vector_ids = vector_ids[0].tolist() scores = scores[0].tolist()
```

Now:

```
vector_ids = [1, 0] scores = [0.88, 0.82]
```

✅ Step: vector_id → item_id mapping

```
item_ids = [vector_map.get(str(v_id)) for v_id in vector_ids]
```

Work it out:

- for v_id = 1 → `vector_map["1"] = "id_bbb"`
- for v_id = 0 → `vector_map["0"] = "id_aaa"`

So:

```
item_ids = ["id_bbb", "id_aaa"]
```

Console output:

```
Top results (vector_id → score → item_id):
  1 → 0.8800 → id_bbb
  0 → 0.8200 → id_aaa
```

✅ Function 3: `fetch_items_by_ids(item_ids)`

Input

```
item_ids = ["id_bbb", "id_aaa"]
```

SQL runs:

```
SELECT item_id, item_name, category, price, veg_flag, city, restaurant_id FROM
phase1_menu_items WHERE item_id = ANY(%s)
```

Postgres returns rows (order NOT guaranteed). Simulated DB output:

```
rows = [ ("id_aaa", "Veg Burger", "Fast Food", 180, "veg", "Delhi", 101), ("id_bbb",
"Paneer Burger", "Fast Food", 190, "veg", "Delhi", 102), ]
```

Now the "preserve order" part:

Build lookup

```
lookup = {r[0]: r for r in rows}
```

So:

```
lookup = { "id_aaa": ("id_aaa", "Veg Burger", "Fast Food", 180, "veg", "Delhi", 101),
"id_bbb": ("id_bbb", "Paneer Burger", "Fast Food", 190, "veg", "Delhi", 102), }
```

Rebuild in FAISS order

```
ordered = [lookup.get(i) for i in item_ids]
```

Since `item_ids = ["id_bbb", "id_aaa"]`, we get:

```
ordered = [ ("id_bbb", "Paneer Burger", "Fast Food", 190, "veg", "Delhi", 102),
("id_aaa", "Veg Burger", "Fast Food", 180, "veg", "Delhi", 101), ]
```

Return:

```
return ordered
```

So back in `main()`:

```
items = ordered
```

✅ Final printing in `main()`

Loop:

```
for row in items: item_id, item_name, category, price, veg_flag, city, restaurant_id = row
```

Console output:

```
Top results (details):
  - Paneer Burger | Fast Food | ₹190 | veg | Delhi | rest_id=102
  - Veg Burger | Fast Food | ₹180 | veg | Delhi | rest_id=101

✅ Sanity search complete.
```

Summary of key variables (as a quick snapshot)

After `load_artifacts()`

```
index = <faiss IndexFlatIP containing vectors> vector_map = {"0":"id_aaa","1":"id_bbb","2":"id_ccc"} metadata = {...}
```

After embedding the query

```
query = "veg burger under 200" q_vec = np.array([[...384 floats...]], dtype=float32)
```

After FAISS search

```
vector_ids = [1, 0] scores = [0.88, 0.82] item_ids = ["id_bbb", "id_aaa"]
```

After DB fetch

```
rows (unordered) = [ ("id_aaa", ...), ("id_bbb", ...) ] lookup = { "id_aaa": (...),
"id_bbb": (...) } items (ordered) = [ ("id_bbb", ...), ("id_aaa", ...) ]
```